Part 3: Slow password hashing

Ethical and authorized use: benchmark only the local demonstration input and use only fictional credentials in the local JSON database.

Run python3 benchmark_kdfs.py and record the median times. The script warms up each implementation and compares SHA-256, PBKDF2, bcrypt, scrypt, and Argon2id with conservative classroom parameters. Values are not universal: production systems must tune memory and work factors for their own hardware and threat model. Argon2id is generally the preferred modern choice; migration constraints may make the other password KDFs relevant.

Try local registration and verification with python3 register_login_demo.py register student01. The JSON database stores metadata, salt, parameters, and a verifier-not plaintext.

Purpose and learning objectives

Part 2 showed that SHA-256 is cheap to guess. A password KDF intentionally makes each verification more expensive while remaining usable for a legitimate login. Modern designs may impose both CPU and memory cost.

After completing this part, you should be able to:

- distinguish a fast general-purpose hash from a password KDF;
- identify the principal work parameters of PBKDF2, bcrypt, scrypt, and Argon2id;
- measure repeated operations and report a median rather than one sample;
- explain why benchmark values are specific to hardware and configuration; and
- inspect a stored authentication record and confirm it contains no plaintext.

Mechanisms being compared

Mechanism	Main classroom parameter	Intended observation
SHA-256	none	Very fast; unsafe baseline for passwords
PBKDF2-HMAC-SHA-256	iterations	Repeats a pseudorandom function
bcrypt	logarithmic cost	Purpose-built and deliberately CPU-expensive
scrypt	n, r, p	Adds a configurable memory cost
Argon2id	time, memory, parallelism	Modern memory-hard password hashing

The benchmark uses one deterministic salt only to make the timing experiment repeatable. Real registration must generate a fresh random salt per password, as register_login_demo.py does.

Tasks

1. Run the three-trial benchmark and record all parameters and median times.
2. Run it again with five trials; compare stability and ordering.
3. Explain why a faster result is not automatically better for password storage.
4. Register a fictional local account and inspect auth_db.json.
5. Verify once with the correct fictional password and once with a different input.
6. Identify the algorithm, iteration count, salt, and verifier stored in JSON.
7. Relate higher legitimate-login cost to higher offline-guessing cost.

Python files and usage examples

‘benchmark_kdfs.py‘

```
cd /workspace/labs/lab01/part3_password_kdfs
python3 benchmark_kdfs.py --trials 3
```

Output has this shape; your timings will differ:

```
SHA-256 (unsafe)             ... ms
PBKDF2-100k                  ... ms
bcrypt-cost-10               ... ms
scrypt-N=2^14                ... ms
Argon2id-32MiB               ... ms
Results are machine-specific; tune parameters for each deployment.
```

The script performs one warm-up followed by the requested number of measured trials and reports their median. Repeat with more trials and compare the ordering, rather than treating one timing as a universal value:

```
python3 benchmark_kdfs.py --trials 5
```

‘register_login_demo.py‘

Use a fictional password that you can enter again, but do not put it on the command line:

```
python3 register_login_demo.py register student01
Password:
registered
```

```
python3 register_login_demo.py verify student01
Password:
verified
```

Entering a different password during verification should print:

```
authentication failed
```

Inspect the generated local database:

```
python3 -m json.tool auth_db.json
```

Confirm that it contains algorithm metadata, iterations, salt, and verifier, but no plaintext password. auth_db.json is ignored by Git.

Use a separate database path when you want an isolated experiment:

```
python3 register_login_demo.py register student02 --db /tmp/lab01-auth.json
python3 register_login_demo.py verify student02 --db /tmp/lab01-auth.json
```

The commands remain interactive; do not place a password in the command line.

Completion criteria and report evidence

You have completed Part 3 when the benchmark reports all five mechanisms, a fictional account can be registered, correct verification succeeds, incorrect verification fails, and no plaintext password appears in JSON. Include a table of parameters and median times, relevant terminal output, and a short tuning recommendation. Do not claim your measurements apply to other machines.

Checkpoint questions

1. How does raising a work factor affect both legitimate verification and an offline attacker?
2. Which demonstrated schemes are designed to consume significant memory?
3. Why should benchmark parameters be tuned on deployment hardware?
4. Why is the SHA-256 timing useful as a baseline but unsafe for storage?